

DeepMindQ Intelligence OS

Comprehensive Audit Report

Actual vs. Planned Gap Analysis

Audit Date	August 14, 2026
Scope	97 screens, 41 API routes, infrastructure, DX
Type	Full application audit
Platform	Next.js 16 + Tailwind v4 + Prisma + Redis

1. Executive Summary

This audit provides a comprehensive assessment of the DeepMindQ Intelligence OS platform, examining 97 screen components, 41 API routes, the full infrastructure stack, and developer experience. The analysis compares the actual implementation against the planned architecture to identify gaps, risks, and prioritized remediation paths. The platform demonstrates strong foundational architecture with a well-designed 7-engine AI pipeline, robust authentication, and a comprehensive Prisma schema. However, significant gaps exist in data connectivity, with 94% of screens relying on hardcoded mock data rather than real API endpoints, and critical infrastructure issues including a dead Tailwind v3 configuration, 330 lines of unused CSS tokens, and 6 stub cron routes that return hardcoded zeros.

97	8	94%	41	6
Total Screens	Production Screens	Mock Data Screens	API Routes	Stub Cron Routes

Severity Distribution

CRITICAL	HIGH	MEDIUM	LOW
7	8	9	8

2. Screen Component Audit

The platform contains 97 screen components across two directories: 85 legacy screens in [/components/screens/](#) and 8 Intelligence OS screens in [/components/intelligence-os/](#). Each screen was evaluated across 8 dimensions: API connectivity, design token usage, animations, interactivity, loading states, error handling, responsive design, and overall quality. The results reveal a significant disparity between the polished appearance and the actual data infrastructure.

2.1 Quality Distribution

Rating	Count	%	Description
PRODUCTION	8	9%	Real API data + loading + error handling + responsive + interactive
DECENT	29	34%	Interactive, good tokens use, some loading; but mock data
MVP	31	36%	Has interactivity and/or loading, but all mock data, no error handling
STUB	17	20%	Read-only display of hardcoded data, minimal/no interaction

2.2 API Connectivity Gap

Only 8 screens connect to real backend APIs. The remaining 89 screens define their data as const arrays at the top of the file and render static content. This means that while the UI appears fully functional with rich data displays, interactive filters, and animated transitions, the actual data flow is entirely simulated. The screens that do connect to APIs are: settings-screen, data-import-screen, intelligence-hub-screen, companies-screen, contacts-screen, users-screen, ai-health-screen, and templates-screen. Notably, the 8 newly built Intelligence OS screens all use mock data with no API integration, making them visually impressive but operationally non-functional.

2.3 Design Token Usage

92% of screens (89/97) properly reference the design token system via `tokens.*` or `var(--ios-*)` CSS custom properties. However, 46 screen files contain at least one hardcoded hex color value, primarily in conditional color logic for severity levels and score thresholds. One screen, `company-profile/intelligence-briefing.tsx`, entirely bypasses the design token system and uses hardcoded Tailwind classes (`text-gray-200`, `bg-white/[0.03]`, `text-violet-400`) that are incompatible with the dark theme. This file also uses raw `fetch()` instead of the project's `fetchApi()` utility, breaking the established error handling and authentication patterns.

2.4 Animation & Interaction Coverage

Dimension	Screens	%	Notes
Animations (Framer Motion)	12	12%	Only Intelligence OS + pipeline screens use motion
Interactive State Management	65	67%	Most screens have filters, tabs, or click handlers
Loading States	56	58%	ScreenSkeleton or Spinner on data fetch

Dimension	Screens	%	Notes
Error Handling (try/catch)	12	12%	Majority will silently fail on API errors
Responsive (mobile breakpoints)	72	74%	Most use grid-cols responsive patterns

3. API Route Audit

The backend consists of 41 API routes organized into 6 categories: authentication (9 routes), business data (10 routes), knowledge graph (7 routes), AI/advisor (1 route), health monitoring (8 routes), and cron jobs (6 routes). The authentication system is exceptionally well-built with proper Zod validation, rate limiting, CSRF protection, timing-safe OTP comparison, and anti-enumeration measures. Health routes are also production-grade. However, all 6 cron routes are stubs, and several significant security gaps exist in the data routes.

3.1 Route Quality Distribution

Rating	Count	Description
PRODUCTION	13	Auth core (login, register, OTP, passwords) + Health routes
DECENT	22	Business data, Knowledge Graph, Advisor - have auth+validation+DB
STUB	6	All cron routes - return hardcoded zeros, no real work

3.2 Security Posture

The authentication layer demonstrates strong security practices including centralized RBAC via `checkApiAuth()`, CSRF protection via `withCsrft()` on state-changing endpoints, timing-safe OTP comparison via `timingSafeCompare()`, anti-enumeration on login with generic error messages and fixed 1-second delay, and security headers applied at the proxy level. However, several gaps exist: the cron secret validation uses simple string equality (`===`) instead of `crypto.timingSafeEqual()`, making it vulnerable to timing side-channel attacks. Twenty authenticated data routes lack per-route rate limiting, relying solely on proxy-level edge limiting. The `/api/knowledge-graph/discover` endpoint uses Zod's `.passthrough()` which allows arbitrary fields to reach downstream library code without validation.

3.3 Critical Finding: Profile Update Crash

The `/api/auth/update-profile` route has a critical bug: its Zod schema allows `phone`, `company`, and `designation` fields, but the Prisma User model does not define these columns. Any profile update attempt that includes these fields will cause a Prisma runtime 500 error. The User model only has: `id`, `email`, `name`, `role`, `passwordHash`, `otpCode`, and `otpExpiresAt`. Either these fields must be added to the Prisma schema and a migration run, or they must be removed from the route's Zod validation schema.

4. Infrastructure & Configuration Audit

The infrastructure layer reveals several significant configuration issues that affect maintainability, bundle size, and developer experience. While the core runtime is functional (TypeScript compiles cleanly, dev server starts in 9.4 seconds, zero build errors), the configuration debt creates confusion and wasted resources across the development lifecycle.

4.1 Design System: Five Competing Token Systems

The `globals.css` file (1,467 lines) contains five overlapping CSS custom property systems that create significant confusion and maintenance burden. The `shadcn/Tailwind` system (`--background`, `--primary`, `--card`) is canonical for UI primitives. The `IOS` system (`--ios-bg-*`, `--ios-border-*`) is the most widely adopted by screen components (40 files). However, the `DMQ` system (~250 properties defined, ZERO used), the `Gold` legacy system (~40 properties, ZERO used), and the `MS6 Locked` token system (~80 properties, near-zero usage) are entirely dead weight. Additionally, confidence colors conflict between systems: `--confidence-high` is `#059669` in one system but `#10b981` in another, despite representing the same semantic concept. Approximately 330 lines of CSS define tokens that no component references.

System	Props	Status	Used By
shadcn/Tailwind (<code>--primary</code> , <code>--card</code>)	~60	Active	UI primitives (Button, Card, etc.)
IOS (<code>--ios-bg-*</code> , <code>--ios-text-*</code>)	~40	Active	40 screen components
Gold Legacy (<code>--color-gold-*</code>)	~40	DEAD	Zero component references
MS6 Locked (<code>--signal-blue-*</code>)	~80	DEAD	Near-zero usage
DMQ Bridge (<code>--dmq-amber-*</code>)	~250	DEAD	Zero component references

4.2 Tailwind v3/v4 Conflict

The project uses Tailwind CSS v4 (via `@tailwindcss/postcss` in `postcss.config.mjs` and `@import "tailwindcss"` in `globals.css`), but retains a v3-style `tailwind.config.ts` file. This v3 config is completely dead – Tailwind v4 ignores it unless explicitly imported via `@config`. The dead config wraps all colors in `hsl()` functions, but the actual CSS variables use hex values, which would produce invalid CSS (`hsl(#2563eb)`). Additionally, `tailwindcss-animate` (a v3 plugin) is installed as a dependency but unused; the project actually uses `tw-animate-css` (v4 compatible). The `tsconfig.json` targets ES2017, unusually low for a Next.js 16 project which defaults to ES2022+, forcing unnecessary downleveling of modern JavaScript features.

4.3 Bundle Size & Unused Dependencies

Package	Size	Severity	Evidence
@xenova/transformers	~50MB	CRITICAL	Never imported anywhere in src/
pdfkit	~2MB	MEDIUM	Never imported in src/
mammoth	~1MB	MEDIUM	Never imported in src/
next-themes	~50KB	LOW	Never imported in src/

Package	Size	Severity	Evidence
tailwindcss-animate	~20KB	LOW	V3 plugin, project uses v4
9 unused shadcn wrappers	~200KB	LOW	drawer, navigation-menu, collapsible, etc.

The `optimizePackageImports` configuration for `recharts`, `framer-motion`, and `lucide-react` is only active when `ANALYZE=true`, meaning tree-shaking for these major bundle contributors is OFF in normal builds. The `images.unoptimized: true` setting in `next.config.ts` disables all Next.js image optimization, which is a footgun for future development if any `next/image` usage is added.

5. Testing Coverage Audit

The project has 30 test files covering 4 categories: end-to-end tests (4 Playwright tests for health, screens, navigation, and auth), API tests (4 tests for settings, organizations, people, and signals), unit tests (8 tests for auth, RBAC, CSRF, validation, and screen smoke tests), intelligence tests (4 tests for knowledge graph, signals reasoning, and ingestion), and AI tests (8 tests for governance, model router, LLM streaming, Redis cache, prompt registry, AI cache layer, and quality gates). While the test infrastructure is comprehensive for backend and AI components, the frontend coverage is critically inadequate.

Gap Area	Coverage	Severity	Details
Intelligence OS Screens	0/8	CRITICAL	No tests exist for any of the 8 new Intel OS screens
Screen Behavior Tests	0	CRITICAL	Smoke tests only verify render, no interaction testing
Integration Tests	0	HIGH	No tests verify data flow between screens and APIs
Component Unit Tests	0	MEDIUM	No tests for shared components (DataTable, StatCard, etc.)
Screens Smoke Coverage	~60/85	LOW	Covers legacy screens but misses Intel OS directory

The primary screen test (screens-smoke.test.ts) only verifies that components render without crashing using `renderToString` and checking `html.length > 0`. It makes zero behavioral assertions about user interactions, data flow, state changes, or error handling. This means a screen could render its initial state correctly but fail completely when a user clicks a button, changes a filter, or submits a form, and no test would catch it. The 8 Intelligence OS screens are completely absent from all test files, representing a significant quality gap for the platform's primary navigation items.

6. Prioritized Action Items

The following 32 findings are organized by severity and estimated remediation effort. Each item includes the specific gap, its impact on the platform, and a concrete remediation step. Items are ordered by business impact: critical items that affect data integrity and security come first, followed by high-priority items that affect functionality, then medium and low items that affect maintainability and developer experience.

6.1 Critical Priority (P0)

These items represent immediate risks to data integrity, security, or core functionality. They should be addressed before any production deployment.

#	Severity	Finding	Impact
1	CRITICAL	Profile update writes non-existent DB fields /api/auth/update-profile allows phone/company/designation but User model lacks these columns. Any update attempt returns Prisma 500 error.	Add fields to Prisma schema + migration, or remove from Zod schema
2	CRITICAL	94% of screens use mock data 89 of 97 screens define const arrays and never call real APIs. The UI appears functional but all data is hardcoded. Business users see realistic but fake information.	Wire screens to existing API routes; start with highest-traffic: sequences, opportunities, pipeline, leads, tasks
3	CRITICAL	6 cron routes are stubs returning zeros data-retention, persistence-evidence, persistence-performance, calibration-runner, backup-verify, and job-processor all have TODO comments and return hardcoded values.	Implement actual cron logic or remove routes and document as unimplemented
4	CRITICAL	Tailwind v3 config is dead, creating confusion tailwind.config.ts is completely ignored by Tailwind v4. It contains invalid hsl(hex) color definitions that would break if the config were ever activated.	Delete tailwind.config.ts, remove tailwindcss-animate dependency

6.2 High Priority (P1)

These items affect security boundaries, data consistency, or significant functionality gaps.

#	Severity	Finding	Impact
1	HIGH	Cron secret comparison not timing-safe All 5 cron routes use === for secret comparison, vulnerable to timing side-channel attacks. An attacker could infer the secret character-by-character.	Replace === with crypto.timingSafeEqual() in validateCronSecret()
2	HIGH	No per-route rate limiting on 20+ data routes Only auth routes have rate limiting. Data routes (organizations, signals, knowledge-graph, advisor/pipeline) have no per-endpoint throttling.	Add rate limits to expensive routes, especially /api/advisor/pipeline
3	HIGH	.passthrough() on discover endpoint allows unvalidated input /api/knowledge-graph/discover uses Zod .passthrough() allowing arbitrary fields to reach downstream discoverRelationships() function.	Remove .passthrough(), explicitly define all allowed fields
4	HIGH	330+ lines of dead CSS tokens in globals.css ~250 --dmq-* tokens, ~40 gold tokens, ~80 MS6 tokens are defined but never referenced by any component. They add 330+ lines of noise.	Delete all --dmq-*, gold, and unused MS6 token blocks

#	Severity	Finding	Impact
5	HIGH	Zero tests for 8 Intelligence OS screens The 8 newly built primary navigation screens (Intelligence Ops, Command Center, etc.) have no test coverage whatsoever.	Add smoke tests + basic behavioral tests for all 8 screens
6	HIGH	company-profile/intelligence-briefing.tsx has broken theming Uses text-gray-200, bg-white/[0.03], text-violet-400 instead of design tokens. Also uses raw fetch() instead of fetchApi().	Replace all hardcoded Tailwind classes with design token references
7	HIGH	@xenova/transformers (~50MB) installed but never used This enormous package is a renamed/deprecated dependency. It bloats node_modules and CI/CD caches significantly.	Remove from package.json
8	HIGH	update-profile uses non-existent Prisma fields The Zod schema allows phone, company, designation fields that do not exist on the User model. Writing these fields causes a Prisma runtime error.	Add fields to schema.prisma User model + run migration

6.3 Medium Priority (P2)

#	Severity	Finding	Impact
1	MEDIUM	Only 12% of screens have error handling 85 screens lack try/catch or error states. If API calls were added without error handling, runtime errors would crash the screen or show no feedback.	Add error boundary fallbacks and try/catch to all screens being wired to APIs
2	MEDIUM	SCREEN_MAP uses string keys instead of ViewId type Record allows typos in screen keys to silently pass. Should be Record for compile-time validation.	Type SCREEN_MAP as Record
3	MEDIUM	optimizePackageImports off by default Tree-shaking for recharts, framer-motion, lucide-react only active when ANALYZE=true. These are major bundle contributors.	Enable optimizePackageImports unconditionally in next.config.ts
4	MEDIUM	require() in screen-map.tsx ESM context ContactDetailBridge uses require(@/lib/store) and require(react) in a use client module. Breaks tree-shaking and static analysis.	Convert to ES import statements
5	MEDIUM	Conflicting confidence color values across token systems --confidence-high is #059669 in one system but #10b981 in another. Same semantic concept, different visual representation.	Consolidate to single canonical value per semantic token
6	MEDIUM	No CORS configuration on any API route Zero Access-Control-Allow-Origin headers. Blocks legitimate cross-origin consumers (mobile apps, third-party integrations).	Add CORS headers or document same-origin assumption
7	MEDIUM	20+ files with explicit any types Screen map, LLM client, Redis client, DB client, and 7 screen files use untyped any.	Progressive typing of function signatures and data shapes
8	MEDIUM	images.unoptimized disables Next.js image optimization next.config.ts has images: { unoptimized: true }, removing all optimization benefits.	Remove unless Docker-specific requirement exists
9	MEDIUM	tsconfig targets ES2017, too low for Next.js 16 Forces downleveling of modern JS features that Next.js 16 assumes are available.	Update target to ES2022 or remove to use Next.js default

6.4 Low Priority (P3)

#	Severity	Finding	Impact
1	LOW	No page-level authentication gate on main page.tsx The app shell renders the full dashboard without checking if a user is authenticated. Any visitor sees the Intelligence OS interface.	Add auth check before rendering AppSidebar + AppHeader
2	LOW	9 unused shaden UI component wrappers drawer, navigation-menu, collapsible, context-menu, hover-card, menubar, toggle, toggle-group, carousel are installed but never imported by any screen.	Remove dead components and their Radix dependencies
3	LOW	Duplicate screen mappings in screen-map.tsx company-workspace, company-detail, company-profile all map to CompanyProfileScreen. accounts and companies both map to CompaniesScreen.	Document intentional aliases or consolidate to canonical mappings
4	LOW	setSidebarCollapsed implemented but not in AppState interface The function exists in the store implementation but is not declared in the TypeScript interface.	Add setSidebarCollapsed and sidebarCollapsed to AppState interface

#	Severity	Finding	Impact
5	LOW	Demo and signup pages were recently fixed to dark theme Both pages now match the Intelligence OS palette, but the signup page still uses a split-screen layout with a light-panel history that may confuse some users.	Consider full-page dark layout for consistency
6	LOW	Pipeline screen Kanban has no mobile breakpoint handling The drag-and-drop Kanban board is desktop-only and would be unusable on mobile devices.	Add a list-view fallback for mobile viewports
7	LOW	Intelligence OS screens lack memoization Only 1 of 8 Intelligence OS screens uses useMemo/useCallback. These are large components with complex state that could benefit from memoization.	Add useMemo/useCallback to expensive computations in Intel OS screens
8	LOW	next-themes installed but never used The theming library is a dependency but the app is hardcoded to dark theme only.	Remove or implement light/dark theme toggle

7. Actual vs. Planned Gap Analysis Matrix

This matrix maps the planned platform capabilities against actual implementation status, providing a clear view of where the platform meets its design intent and where significant gaps remain. Each capability area is assessed on a maturity scale from Not Started to Production.

Capability Area	Status	Complete	Gap Description
Signal Detection & Monitoring	Production	85%	Real API, filters, DataTable, pagination. Needs more signal types.
AI-Powered Scoring	MVP	30%	Scoring config screen exists but no real AI scoring pipeline connected to UI.
Revenue Intelligence	MVP	25%	Screens built with mock data. No real revenue data pipeline.
Pipeline Management	Decent	60%	DnD Kanban works with mock data. Real pipeline API exists but not wired.
Knowledge Management	MVP	35%	Knowledge library and workspace built. No real knowledge graph queries.
Sequence Automation	Decent	55%	Full sequence builder UI. Uses mock sequence data, no send integration.
AI Command Center	MVP	40%	Monitoring dashboard with mock metrics. Real AI health API exists.
Intelligence Search	Decent	50%	Search UI recently built. No real search backend (no Elasticsearch).
Intelligence Briefing	Decent	45%	Briefing viewer built. Generate button is cosmetic only.
Authentication & RBAC	Production	95%	Full OTP flow, CSRF, rate limiting, RBAC matrix.
Health Monitoring	Production	90%	7 health endpoints, liveness, readiness, Prometheus metrics.
Cron Operations	Stub	5%	6 routes exist but all return hardcoded zeros.
Data Import	Production	75%	File upload, validation, history. Connected to real ingestion API.
Settings Management	Production	80%	Full CRUD with real API. Env-based config.

8. Recommended Remediation Roadmap

Based on the findings, the following phased roadmap prioritizes the highest-impact items first. Phase 1 addresses critical security and data integrity issues. Phase 2 connects the UI to real data. Phase 3 cleans up technical debt. Phase 4 enhances quality and coverage.

Phase 1: Security & Integrity (Week 1-2)

- ââ Fix update-profile Prisma schema mismatch (P0) -- add User fields or remove from Zod
- ââ Replace === with crypto.timingSafeEqual() in cron secret validation (P1)
- ââ Remove .passthrough() from /api/knowledge-graph/discover endpoint (P1)
- ââ Add per-route rate limiting to data endpoints (P1)
- ââ Add authentication gate to main page.tsx (P3)

Phase 2: Data Connectivity (Week 3-6)

- ââ Wire top 10 screens to real APIs: sequences, opportunities, pipeline, leads, tasks, intelligence ops, command center, activation workspace, knowledge workspace, capability workspace
- ââ Implement real search backend (Elasticsearch or PostgreSQL full-text) for Intelligence Search
- ââ Connect Intelligence Briefing to real /api/briefings endpoint with generation
- ââ Add error handling (try/catch + fallback UI) to all newly wired screens
- ââ Replace mock data generation (Math.random(), getMockSignals) with real data fetching

Phase 3: Technical Debt Cleanup (Week 7-8)

- ââ Delete dead Tailwind v3 config and tailwindcss-animate dependency
- ââ Remove 330+ lines of unused CSS tokens (--dmq-*, gold, unused MS6)
- ââ Remove unused dependencies: @xenova/transformers, pdfkit, mammoth, next-themes
- ââ Consolidate design token systems to 2 (shadcn + ios), deprecate the rest
- ââ Fix conflicting confidence color values across token systems
- ââ Convert require() to import in screen-map.tsx
- ââ Type SCREEN_MAP as Record
- ââ Update tsconfig target to ES2022

Phase 4: Quality & Coverage (Week 9-12)

- ââ Add tests for all 8 Intelligence OS screens (smoke + behavioral)
- ââ Add integration tests for screen-to-API data flow
- ââ Fix company-profile/intelligence-briefing.tsx theming

